

A Sovereign Coding Agent on macOS — PI in an Apple Container; Zero npm on the Host



Michael Hannecke

Follow

11 min read · May 19, 2026



119



1



Made with Google Nano Pro

Run a local AI coding agent that reads code and runs shell commands — without handing it your filesystem, your credentials, or a Node toolchain on your work

machine. This is a companion to the MLX-Swift series. Link to the repo at the bottom.

What you'll build:

The `pi` coding agent (link at the bottom) inside a disposable Apple `container` micro-VM, talking to a local MLX-Swift model on the host. No Node, no npm, no agent binary on macOS. One project directory in, nothing else reachable, host unchanged on exit.

Why put a coding agent in a container at all?

A modern coding agent is, by design, a program that reads your files, runs shell commands, and installs whatever it decides it needs. That is what makes it useful — and what makes it a liability on a work machine in a regulated context.

Run it directly on the host and you implicitly grant it:

- **Your entire filesystem.** Not just the project: `~/.ssh`, `~/.aws`, `~/.config/gcloud`, browser profiles, every other client's repo on the same disk.
- **A live npm/Node toolchain.** One `npm install` pulls transitive dependencies whose post-install scripts run with your privileges. On a host that also holds production credentials, that blast radius is unacceptable.
- **Network reach as you.** Any outbound call leaves with your identity and your routes.

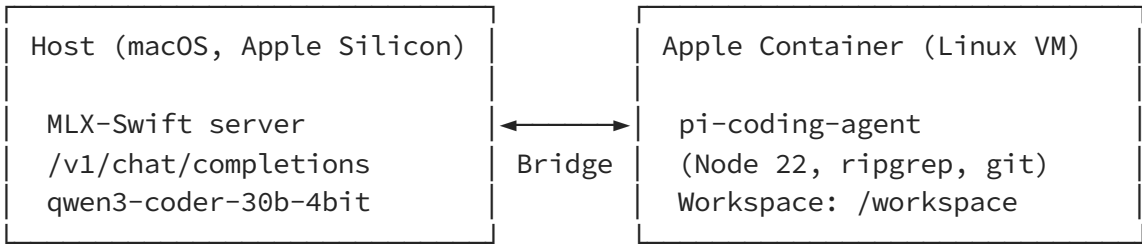
This is not a concern the tooling ignores. pi's own maintainer is explicit that the agent has no permission popups by design, and that the intended way to contain it is to run it in a container. We are following the tool's security model, not fighting it.

For a regulated audience the question is not “is the model good.” It is “**what exactly can this process touch, and can I prove it.**” Sovereignty here means three concrete things: the model runs locally, no prompt or code leaves the machine, and the agent runtime is confined to a boundary you defined. Two decisions follow:

1. **No Node, no npm, no pi binary on the host.** The agent lives only inside an image; the host runs exactly one thing — the local model.
2. **The agent runtime is sandboxed and disposable.** It sees one project directory and one model endpoint. Throw the container away after the session; the host is byte-for-byte unchanged.

Inference stays **native on the host**: MLX-Swift needs Apple Silicon's Metal/ANE, which a Linux VM does not expose. That constraint isn't something to work around — it produces a clean split:





Inference on the host (it has to be), tool-calling sandbox in the container, and the two talk only over the container bridge.

Why Apple container , not Docker, on Apple Silicon

Docker Desktop is the reflex answer. For this use case Apple’s own container CLI is the better tool, for non-cosmetic reasons:

Concern	Docker Desktop on macOS	Apple container
Isolation	One shared Linux VM; every container is a process inside it. A kernel escape affects all containers.	One lightweight VM per container, on Apple's Virtualization.framework. The boundary is a VM, not a shared kernel.
Footprint	Heavy always-on VM plus a background daemon.	Per-container micro-VMs that start in well under a second and vanish with the container.
Daemon / licensing	Long-running daemon; commercial licensing terms for larger orgs.	No third-party daemon, no extra licensing — an Apple-shipped open-source tool.
Apple Silicon fit	arm64 works, but it's a generic stack bolted onto macOS.	Built for Apple Silicon and recent macOS.
Networking	Docker-style port publishing.	Each container gets its own IP on a private bridge; the agent reaches the host <i>outbound</i> over the gateway — no port-publishing dance.

The decisive point for a sovereignty review is one sentence: **each container is its own VM**. That is far stronger, and far easier to defend, than “containers share a kernel.” You point at a virtualization boundary, not at namespace hardening — and every other benefit below is downstream of that one fact.

Scope note. The model server here is MLX-Swift. Because `models.json` only points at an OpenAI-compatible URL, the *same container* works unchanged against any local server speaking OpenAI chat-completions with tool-calling (Ollama, llama.cpp, ...) — a one-line `baseUrl` change.

Prerequisites

- **macOS 26 (Tahoe) on Apple Silicon, recommended.** `container` technically runs on macOS 15, but its networking is significantly limited there and maintainers primarily support current macOS — and this whole setup lives or dies on container-to-host networking. Treat macOS 15 as unsupported here.
- A local model server **on the host** with an OpenAI-compatible `/v1/chat/completions` endpoint **and tool-calling**, serving `qwen3-coder-30b-a3b-instruct-4bit`. Standing that up is the prior host-only part of this series; here it's a prerequisite, verified in Step 2.
- This repository checked out locally (it ships the `Containerfile`, `pi-config/`, and `scripts/`).
- **macOS Local Network permission available to grant** (recent macOS gates local traffic behind a privacy prompt — Step 1).
- **No Node and no npm on the host.** That is the point.

Step 1: Install the Apple container CLI

container ships as a signed macOS installer from Apple's open-source project.

1. Download the latest release `.pkg` from the `apple/container` GitHub releases page — current release, not a pinned version, not a source tarball.
2. Run the standard macOS installer.
3. (Optional) If you already have apple container installed, update to the latest version:

```
# stop running container service
container system stop

# run update script
/usr/local/bin/update-container.sh
```

4. Start the runtime and confirm the CLI answers:

```
container system start
container --version
```

A printed version means the runtime is up. If `container system start` fails, you are almost certainly not on a supported macOS / Apple Silicon

combination — a hard requirement of the virtualization stack, no workaround.

Get Michael Hannecke's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Grant Local Network access now. On first run, macOS prompts for local-network access; allow it for the container runtime (and for your terminal when testing). Skipping this causes the most confusing failure later: traffic silently dropped, no error, empty reply. If you dismissed the prompt, fix it under *System Settings* → *Privacy & Security* → *Local Network*. Note that nothing here installed a third-party daemon or licensing agent — the runtime is an Apple-signed component, which is the install story a review will ask about.

Step 2 : Confirm the local model is reachable

Inference cannot move into the container (no Metal/ANE in a Linux VM), so the model server stays on macOS. What the container needs from the host:

- An HTTP endpoint speaking **OpenAI chat-completions**. (running Ollama, LMStudio, oMLX or mlx-lm — whatever suits you)
- `qwen3-coder-30b-a3b-instruct-4bit` loaded.
- **Tool-calling enabled** — the agent is inert without it.
- The server bound to `0.0.0.0:8080` , **not** only `127.0.0.1:8080` . This is the most common reason "it can't reach the model" later.

Verify from the host (exact path depends on your server — the point is that it answers the OpenAI protocol):

```
curl -s http://127.0.0.1:8080/v1/models
```

A model list means the host side is ready. Keep port 8080 consistent below.

Step 3 — The repository layout

```
.
├── Containerfile                                # node:22-bookworm-slim + pi i
├── pi-config/
│   ├── AGENTS.md                               # global agent rules (containe
│   ├── models.json                             # provider + model definition
│   └── extensions/
│       └── protected-paths/
│           └── index.ts                         # tool-call guardrail for sens
└── scripts/
    ├── build.sh                                # container build
    └── run.sh                                  # container run with the right
```

`pi-config/` is mounted at runtime as the agent's config directory. Its `sessions/`, `cache/`, and `logs/` subdirectories are produced by pi during a session and are git-ignored — runtime artifacts, not configuration.

Step 4: The Containerfile, explained

The entire image: a minimal Node 22 base, the few CLI tools pi's bash tool uses, pi installed globally, a non-root user.

```
# Pi Coding Agent inside an Apple container.
#
# Minimal Node image; pi installed globally, tools for the
# bash tool-call (find, grep, rg) available, /workspace as the
# mount target for the respective project.

FROM node:22-bookworm-slim

RUN apt-get update \
  && apt-get install -y --no-install-recommends \
    git \
    ripgrep \
    ca-certificates \
    iproute2 \
  && rm -rf /var/lib/apt/lists/*

RUN npm install -g @mariozechner/pi-coding-agent

ARG PI_UID=1000
ARG PI_GID=1000
# node:22 already ships a 'node' user/group at UID/GID 1000; remove it so the
# 'pi' user can own that id range, then create pi.
RUN userdel --remove node 2>/dev/null || true \
  && groupdel node 2>/dev/null || true \
  && groupadd --gid ${PI_GID} pi \
  && useradd --uid ${PI_UID} --gid ${PI_GID} --create-home --shell /bin/bash pi

USER pi
WORKDIR /workspace

# pi reads ~/.pi/agent/* at runtime; the directory is mounted via a volume.
ENTRYPOINT ["pi"]
```

- **git, ripgrep, iproute2, ca-certificates** — git/rg back pi's bash tool for read-only inspection; iproute2 gives ip route (needed in Step 6); ca-

certificates for TLS. A small surface is part of the security argument.

- **npm install -g @mariozechner/pi-coding-agent** — the only npm install in the story, at build time, inside the image, never on the host.
- **Non-root pi user** — the agent never runs as root inside the VM.
- **ENTRYPOINT ["pi"]** — the container is the agent; anything appended on run becomes pi's arguments.

Step 5: Build the image

```
./scripts/build.sh
```

A thin, reproducible wrapper:

```
#!/usr/bin/env bash
# Builds the pi-coding-agent image for the Apple container.
set -euo pipefail

IMAGE_TAG="${IMAGE_TAG:-pi-coding-agent:local}"
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"

container build \
  --tag "$IMAGE_TAG" \
  --file "$REPO_ROOT/Containerfile" \
  "$REPO_ROOT"
```

It produces `pi-coding-agent:local` (override with `IMAGE_TAG=...`). Success here means exactly one thing: **container build completes without error**.

Step 6: Find the host bridge IP and wire it into `models.json`

From inside the container the host is reachable via the bridge's default gateway. The address is **environment-dependent** — it varies by container version, so discover it instead of assuming a subnet:

```
sc container run --rm --entrypoint sh pi-coding-agent:local -c "ip route | awk
```

The printed address is the host's IP *as seen from inside the container*. The shipped `pi-config/models.json` uses a common default:

Open in app ↗

Sign up

Sign in

Medium

Search

Write



```
"api": "openai-completions",
"apiKey": "not-required",
"models": [
  {
    "id": "<yourlocalpath>/models/gemma-4-26b-a4b-it-4bit",
    "name": "mlx-local/gemma4-instruct"
  }
]
}
```

If the discovered gateway differs from `192.168.64.1`, edit `providers.mlx-local.baseUrl` — keep the `:8080/v1` suffix (no `/chat/completions`; `pi` appends the rest).

- Adapt “ID”: “<yourlocalpath” to your local setup . By default it’s most often in “`~/cache/huggingface/hub`”
- **apiKey: "not-required"** — a local server needs no secret. No credential exists, so none can leak.
- **toolCalling: true** — this is what lets the agent edit files and run commands, not just chat.
- **contextWindow / maxOutputTokens** — sized to the MLX server's limits; don't raise past what the host serves.

Step 7: Guardrails: `AGENTS.md` and `protected-paths`

Two pieces of the mounted config turn “an agent in a box” into “an agent in a box that behaves.”

`pi-config/AGENTS.md` is loaded into every session as the operating contract: the session runs in an Apple container, the host is not directly reachable, file operations only affect `/workspace`, the model is reached only over the bridge, no external calls or telemetry without explicit instruction, and tool discipline (`read before edit`, `write only` for new files, no `npm install` without confirmation). Sovereignty as standing policy, not hope.

`pi-config/extensions/protected-paths/index.ts` is a defense-in-depth backstop. Even though the container only sees `/workspace`, the moment anyone *also* mounts a host path, this extension hooks `pi's tool_call` event

and forces a confirmation (or hard-denies) for sensitive directories and patterns:

```
const PROTECTED_DIRS = [  
  path.join(os.homedir(), ".ssh"),  
  path.join(os.homedir(), ".aws"),  
  path.join(os.homedir(), ".config/gcloud"),  
  "/run/secrets",  
  "/etc",  
];  
  
const PROTECTED_PATTERNS: RegExp[] = [  
  /\.env(\.|$)/,  
  /credentials\.json$/,  
  /id_rsa(\.|$)/,  
  /id_ed25519(\.|$)/,  
  /\.pem$/,  
  /\.p12$/,  
];
```

It inspects the `read / write / edit` target path *and* scans `bash` commands for the same — so `cat ~/.ssh/id_rsa` is caught as readily as a direct read. The container is the strong boundary; this is the seatbelt for the day someone widens a mount.

Step 8: Run the agent

```
PROJECT_DIR=~/projects/your-repo ./scripts/run.sh --model mlx-local/gemma4-instr
```

`run.sh`, deliberately boring and explicit:

```
#!/usr/bin/env bash
# Startet pi in einem Apple-Container.
set -euo pipefail

IMAGE_TAG="${IMAGE_TAG:-pi-coding-agent:local}"
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
PROJECT_DIR="${PROJECT_DIR:-$(pwd)}"

if [ ! -d "$PROJECT_DIR" ]; then
    echo "PROJECT_DIR='$PROJECT_DIR' existiert nicht." >&2
    exit 1
fi

container run \
    --rm \
    --interactive \
    --tty \
    --volume "$REPO_ROOT/pi-config:/home/pi/.pi/agent" \
    --volume "$PROJECT_DIR:/workspace" \
    --workdir /workspace \
    "$IMAGE_TAG" \
    "$@"
```

Exactly two things cross the boundary, by mount and nothing else:

- `pi-config/` → `/home/pi/.pi/agent` — provider config, `AGENTS.md`, the `protected-paths` extension.
- `$PROJECT_DIR` → `/workspace` — *only* the project you point it at. Unset, it defaults to the current directory, so always set it deliberately.

`--rm` discards the VM and its writable layer on exit. The only persistent traces live inside the project and `pi-config/sessions|cache|logs` (git-ignored). Everything after the image name is passed straight to pi via the entrypoint.

Step 9: Smoke test: prove the boundary works

A session “works” when both halves are confirmed:

1. **The agent reached the model.** Ask something that forces a model round-trip; a coherent answer means the container resolved the bridge IP and the host responded. If it hangs or errors, the cause is almost always one of the first two troubleshooting items.
2. **The agent uses tools, but only inside `/workspace`.** Ask it to list and read a project file (exercises `bash / read` against the mount), then ask it to touch something outside `/workspace`: it should refuse on the `AGENTS.md` contract, with `protected-paths` catching anything that slips past.

Both hold → a sovereign setup: local model, no npm/Node on the host, agent confined to one project inside a per-container VM, clean host the moment you exit.

Troubleshooting (the things that actually go wrong)

- **Local Network permission not granted.** Symptom: requests hang or fail with no error and an empty reply, nothing obviously misconfigured. macOS blocks local-network traffic by default. *System Settings* → *Privacy & Security* → *Local Network* — enable the container runtime (and the requesting app), then fully quit and reopen that app. On recent macOS this is the most common first-run failure.
- **Host bound to loopback.** The container is a separate VM; it cannot reach host `127.0.0.1`. Bind the model server to `0.0.0.0:8080` and re-test.

- **Wrong bridge IP.** `192.168.64.1` is only a default. Re-run the `ip route` discovery from Step 6 and use the actual gateway. Don't assume a subnet.
- **Container files not owned by your macOS user.** Expected: the container writes as UID 1000, your host user is typically UID 501. In the pi workflow (edits go through the `edit` tool) this is acceptable — just know *why*.
- **The agent answers but never edits.** Check `toolCalling: true` *and* that the host server genuinely supports tool-calling. A chat-only model looks like it works and silently does nothing.

What you've gained

- **No npm, no Node, no pi on the host.** The only npm install happened at image build time, inside the VM.
- **A per-container VM boundary,** not shared-kernel namespaces — the isolation claim you can defend without arguing namespace hardening.
- **Local inference, local data.** Nothing leaves the machine; no external network reach by policy, no credential to leak by configuration.
- **Disposable and reproducible.** `--rm` plus a pinned `Containerfile: rebuild` it identically tomorrow, throw it away tonight.

For a regulated / DACH context that is the whole pitch: the model is yours, the data stays put, and “what can this agent touch” has a precise, demonstrable answer.

Deliberately out of scope

Each is its own follow-up, not something to bolt on here:

- Standing up the MLX-Swift host server (the prior host-only part of the series).
- Multi-project / concurrent sessions and image lifecycle.
- Network egress filtering at the VM level (here egress is by agent policy, not yet enforced at the bridge).
- CI / headless agent runs.
- Pinning a container CLI version and supply-chain attestation for the image.

Github Repos with all files (archived):

<https://github.com/michaelhannecke/pi-container>

Pi Coding Agent

Apple Container

Ai Coding Agent

Apple Silicon



Written by Michael Hannecke

916 followers · 97 following

Follow

AI Strategist @ bluetuple.ai | Secure AI systems and practical AI applications.
Let's connect on LinkedIn: <https://www.linkedin.com/in/michaelhannecke/>

